

Lightweight and Reproducible Cassava Leaf Disease Classification: A Controlled Study of Architecture and Training Design on Field Images

Marko Miovski

April 28, 2026

Abstract

Cassava leaf disease classification is an important agricultural image-recognition problem because cassava is a major food-security crop and disease detection can support crop monitoring and farmer decision-making. This study examines cassava disease classification from a lightweight and reproducible modeling perspective rather than from a maximum-complexity or state-of-the-art competition perspective. Using the Kaggle Cassava Leaf Disease Classification dataset, I conducted two controlled experiments on five-class cassava leaf image classification. The first experiment compared three related CNN architectures, namely a baseline CNN, a residual CNN, and a residual CNN with lightweight channel attention. The residual attention CNN achieved the strongest architecture result, with validation macro F1 of 0.7260 and validation accuracy of 0.8427, while adding only about 1.0% more parameters than the residual CNN. The second experiment fixed the residual attention CNN and compared three training strategies: simple augmentation, stronger field-oriented augmentation, and stronger augmentation with weighted cross-entropy. The simple augmentation setup performed best, reaching validation macro F1 of 0.7449. The final selected configuration, a residual attention CNN trained with simple augmentation, achieved test accuracy of 0.8589, test macro F1 of 0.7365, and test loss of 0.4183. Class-level performance remained uneven, with Cassava Mosaic Disease (CMD) achieving the highest recall at 0.9696 and Cassava Bacterial Blight (CBB) having the lowest recall at 0.5092, along with notable confusion between CBB and Healthy leaves. Overall, the results suggest that targeted lightweight architectural refinement was more useful than adding training complexity, but they also show that aggregate performance does not fully capture deployment reliability across disease classes.

Contents

1	Introduction	4
1.1	Motivation: Cassava Disease Classification as a Field-Image Problem	4
1.2	Technical Difficulty of Field-Image Classification	4
1.3	Gap in Prior Work	4
1.4	Study Objective and Contributions	5
2	Related Work	5
2.1	Cassava Leaf Disease Classification on Field Images	5
2.2	Lightweight CNN Architectures for Efficient Vision	6
2.3	Augmentation and Robustness in Plant Disease Recognition	6
2.4	Residual Connections and Channel Attention in CNNs	6
3	Methods	7
3.1	Dataset and Task Formulation	7
3.2	Data Validation, Exploratory Findings, and Split Construction	7
3.3	Base Preprocessing and Data Pipelines	9
3.4	Model Family and Architecture Design	11
3.4.1	Baseline CNN	11
3.4.2	Residual CNN	12
3.4.3	Residual Attention CNN	12
3.4.4	Controlled Architecture Progression	13
3.5	Training Protocol	13
3.6	Evaluation Metrics	14
4	Experiments	15
4.1	Overview of Experimental Design	15
4.2	Experiment 1: Architecture Ablation	15
4.3	Experiment 2: Training-Strategy Ablation	16
4.4	Experimental Questions	17
5	Results and Discussion	17
5.1	Experiment 1: Architecture Ablation Results	17
5.2	Experiment 1 Discussion	19
5.3	Experiment 2: Training-Strategy Ablation Results	19
5.4	Final Test Performance	21
5.5	Class-Level Behavior	21

5.6	Overall Discussion and Synthesis	23
6	Conclusion	24
6.1	Main Takeaway	24
6.2	Limitations and Unanswered Questions	24
6.3	Future Work	25
6.4	Ethical and Social Implications	25

1. Introduction

1.1 Motivation: Cassava Disease Classification as a Field-Image Problem

Cassava leaf disease classification is motivated by a practical agricultural problem rather than by image classification alone. Cassava is a major food-security crop in Africa, with the Food and Agriculture Organization of the United Nations noting that more than 70 million people in Africa depend on it [2]. Cassava is also valued because it can produce acceptable yields on marginal land, making threats to cassava production especially important for vulnerable communities [2]. When pests or diseases reduce cassava yield, the consequences can affect food supply, farmer income, and local agricultural stability. This makes image-based disease detection a useful direction to study. Prior cassava work notes that at least 80% of smallholder farmer households in Sub-Saharan Africa grow cassava and that smartphone access creates a possible pathway for monitoring disease directly from farm photos [8]. In this setting, a lightweight model that can classify cassava leaf images may eventually support earlier disease recognition, especially where expert diagnosis is difficult to access.

The motivation for this study follows from that deployment context. A cassava disease classifier intended for field use should not only perform well in aggregate; it should also be lightweight, reproducible, and understandable enough to study systematically. Highly complex models may achieve strong benchmark performance, but they are not automatically the most practical or informative systems for agricultural use. This study therefore focuses on controlled, lightweight CNN design choices rather than on maximizing architecture complexity.

1.2 Technical Difficulty of Field-Image Classification

Field-image classification is difficult because agricultural images are naturally variable. Unlike controlled laboratory images, field photos may include inconsistent lighting, cluttered backgrounds, different camera distances, changing viewpoints, leaf occlusion, and variation in disease visibility. A model must learn disease-relevant visual patterns while ignoring many details that are not directly related to the leaf condition.

Cassava disease classification also requires attention to class-sensitive behavior. Some disease categories may be visually easier to identify than others, and some classes may appear more frequently in the data than others. In this setting, aggregate accuracy can give an incomplete picture of model reliability. This means a classifier may perform well overall while still missing a weaker disease class or confusing a diseased leaf with a healthy one.

For this reason, the study evaluates models using macro F1-score, per-class recall, and confusion matrices in addition to accuracy. These metrics are important because the intended problem is not merely to classify many images correctly on average, but to understand whether a lightweight model behaves reliably across disease categories.

1.3 Gap in Prior Work

Prior work has shown that cassava disease classification is feasible, and much of the literature has focused on improving predictive performance through CNN-based, attention-based, or other specialized architectures [9, 8, 10]. These studies demonstrate that image-based cassava disease recognition can work, but they do not fully answer the central question of this study, namely which lightweight

architectural and training choices are most useful under realistic field-image conditions.

That question matters because a high-performing model is not automatically the most reproducible, efficient, or practical model to study. A larger architecture or more complicated training strategy may improve one metric while adding computational cost, reducing interpretability, or making the pipeline harder to reproduce. For field-oriented agricultural applications, the tradeoff between performance and complexity is part of the research problem.

This study addresses that gap by treating cassava disease classification as a controlled empirical study rather than as a state-of-the-art competition task. Instead of comparing many unrelated high-capacity models, it studies a small family of related CNNs and a small set of training strategies while holding the rest of the pipeline fixed. This makes it easier to interpret which design choices are responsible for performance changes.

1.4 Study Objective and Contributions

The objective of this study is to evaluate lightweight CNN design choices for cassava leaf disease classification under a controlled experimental framework. The study focuses on predictive performance, class-sensitive behavior, model complexity, reproducibility, and practical deployment relevance. The central question is not only which configuration obtains the highest score, but which choices provide the clearest tradeoff between performance and complexity.

This study makes four main contributions. First, it provides a controlled architecture ablation comparing a baseline CNN, a residual CNN, and a residual CNN with lightweight channel attention. This experiment tests whether residual learning and channel-wise feature recalibration improve performance enough to justify additional model complexity. Second, it provides a controlled training-strategy ablation comparing simple augmentation, stronger field-oriented augmentation, and imbalance-aware training. This experiment tests whether additional training complexity improves generalization and class-sensitive performance after the architecture is fixed. Third, the study uses a reproducible experimental pipeline with fixed splits, shared preprocessing, consistent training settings, and train-only normalization statistics. Fourth, it evaluates models using validation macro F1-score, accuracy, per-class recall, confusion matrices, and parameter count. Together, these contributions support the broader goal of understanding lightweight cassava disease classification as a performance-complexity tradeoff rather than as a search for the largest possible model.

2. Related Work

2.1 Cassava Leaf Disease Classification on Field Images

Prior work has established cassava leaf disease classification as a feasible and important computer vision task. Ramcharan et al. studied image-based cassava disease detection using deep learning, showing that CNN-based models could support recognition of cassava disease symptoms from leaf images [9]. The iCassava work further framed cassava disease recognition as a fine-grained visual categorization problem connected to real agricultural monitoring needs in Sub-Saharan Africa [8]. Sambasivam and Opiyo also studied cassava disease classification with CNNs in the presence of class imbalance, which is relevant because uneven class frequencies can affect both training behavior and evaluation [10]. Together, these studies motivate cassava disease recognition as a meaningful application area for deep learning.

This study builds on that literature but focuses on a different question. Prior cassava disease classification work often emphasizes whether strong predictive performance is possible. In contrast, this study uses the cassava field-image setting to examine how lightweight architectural and training choices affect performance, reproducibility, and model complexity. The goal is not to claim a new state-of-the-art cassava classifier, but to run a controlled comparison that makes the effects of specific design choices easier to interpret.

2.2 Lightweight CNN Architectures for Efficient Vision

Efficient CNN architectures are an important part of deployment-oriented computer vision. MobileNetV2 introduced inverted residuals and linear bottlenecks as a way to improve the accuracy-efficiency tradeoff for mobile and resource-constrained settings [11]. ShuffleNet similarly focused on efficient convolutional design for mobile devices, using architectural choices intended to reduce computational cost while preserving useful representational capacity [14]. EfficientNet studied model scaling as another way to balance accuracy and efficiency across CNN architectures [12]. These works show that model design is not only about predictive accuracy, but also about how much computation and model capacity are required to achieve it.

This study is related to that efficiency-oriented literature, but it does not attempt to introduce a new general-purpose lightweight CNN architecture. Instead, it studies a small custom model family within a specific agricultural field-image task. This narrower scope is intentional. By comparing a baseline CNN, a residual CNN, and a residual attention CNN under fixed experimental conditions, the study examines whether modest architecture changes are useful for cassava disease classification and whether their performance gains justify their added parameter cost.

2.3 Augmentation and Robustness in Plant Disease Recognition

Plant disease recognition often depends on how well models generalize across image conditions. Hasan et al. review deep learning for plant disease recognition and discuss the broader role of deep models in agricultural disease classification [4]. Other work has studied augmentation and robustness strategies more directly. Cap et al. proposed LeafGAN as a data augmentation method for practical plant disease diagnosis, reflecting the importance of generating or transforming training data to improve disease recognition [1]. Gui et al. studied automatic field plant disease recognition, which is relevant to the challenge of building models that can work under more realistic field-image variation [3].

This study uses that literature as motivation for the training-strategy ablation. Field images can vary in lighting, orientation, color, blur, and image quality, thereby making augmentation a reasonable strategy to test. However, this study does not assume that stronger augmentation will automatically improve performance. Instead, it compares a simple augmentation setup against a stronger field-oriented augmentation setup and a weighted-loss setup. This controlled design makes it possible to evaluate whether additional training complexity actually improves cassava disease classification under the chosen experimental conditions.

2.4 Residual Connections and Channel Attention in CNNs

Residual connections and channel attention are widely used CNN design ideas. He et al. introduced deep residual learning, where shortcut connections help networks learn residual mappings and improve optimization in deeper convolutional models [5]. Hu et al. introduced squeeze-and-excitation networks,

which use channel-wise feature recalibration to emphasize informative channels and suppress less useful ones [6]. Woo et al. proposed the convolutional block attention module, extending the idea of attention in CNNs through channel and spatial attention mechanisms [13]. These works motivate residual learning and attention as reasonable architectural components to test in a CNN classifier.

This study uses those ideas in a deliberately limited and controlled way. The residual CNN tests whether skip connections improve over a plain CNN baseline in the cassava field-image setting. The residual attention CNN then tests whether lightweight channel attention provides additional value beyond residual learning alone. Rather than adopting a large pretrained architecture or a complex attention system, the study evaluates these components inside a small custom CNN family so that the performance-complexity tradeoff remains clear.

3. Methods

3.1 Dataset and Task Formulation

This study examines cassava leaf disease classification as a supervised five-class image classification problem. The dataset used is the Kaggle Cassava Leaf Disease Classification Competition dataset, which contains 21,397 labeled cassava leaf images collected under realistic field conditions in Uganda, with the majority of images crowdsourced from farmers and then annotated by experts at the National Crops Resources Research Institute (NaCRRI) in collaboration with the AI Lab at Makerere University, Kampala, Uganda [8, 7]. Each image is assigned one of five class labels: Cassava Bacterial Blight (CBB), Cassava Brown Streak Disease (CBSD), Cassava Green Mottle (CGM), Cassava Mosaic Disease (CMD), or Healthy. The goal of the model is therefore to learn a mapping from an input cassava leaf image to one of these five disease or health categories.

The dataset is aligned with the motivation of the study because it consists of field-acquired cassava images rather than controlled laboratory images. This distinction is important because the intended use case is not a clean image-recognition benchmark, but a practical agricultural setting in which leaf images may vary in lighting, background, orientation, and visual quality. Since the overall goal of this study is to evaluate lightweight and reproducible models for realistic crop-disease recognition, the dataset provides an appropriate test case for studying whether modest CNN design choices can improve performance under field-image conditions.

Although the Kaggle competition includes a test-image directory, the publicly available test set in the downloaded competition files contained only one visible image. For this reason, the study does not rely on the competition test set for final evaluation. Instead, the labeled training data were split into fixed train, validation, and test partitions. This allowed all experiments to use the same held-out validation and test sets, thereby making the model comparisons reproducible and ensuring that the final test results were evaluated only after validation-based model selection.

3.2 Data Validation, Exploratory Findings, and Split Construction

Before modeling, the dataset was checked to ensure that the labels, image files, and class mappings were usable for a controlled experimental study. The metadata file was loaded into a dataframe, image paths were reconstructed from the provided image identifiers, and each listed image file was checked for existence. The label mapping file was also loaded and used to attach class names to the numeric labels. These checks confirmed that the dataset contained 21,397 rows and 21,397 unique image identifiers,

with no null image IDs, no null labels, no unmapped class names, and no missing image files. A sample corruption check was also performed by opening and verifying a random sample of 100 images, with no corruption issues found in that sample.

After these checks, a minimally cleaned dataframe was created. Since the technical validation found no missing labels or missing files, no rows were removed during this cleaning step, and the final usable dataset still contained 21,397 images. This was important for reproducibility because the final dataset used for splitting and modeling was identical in size to the validated metadata table rather than being affected by undocumented filtering choices.

The exploratory analysis showed two dataset properties that directly affected the experimental design. First, the class distribution was imbalanced, with CMD representing the dominant class, as shown in Figure 1. This imbalance motivated the use of macro F1-score as the primary validation metric because macro F1 gives equal weight to each class rather than allowing the majority class to dominate the evaluation. It also motivated the inclusion of an imbalance-aware loss function in Experiment 2. Second, an image-size analysis showed that all raw images shared the same dimensions and aspect ratio: 800 pixels wide by 600 pixels high, corresponding to a fixed 4:3 aspect ratio. This finding simplified preprocessing because the pipeline did not need to handle mixed raw image shapes.

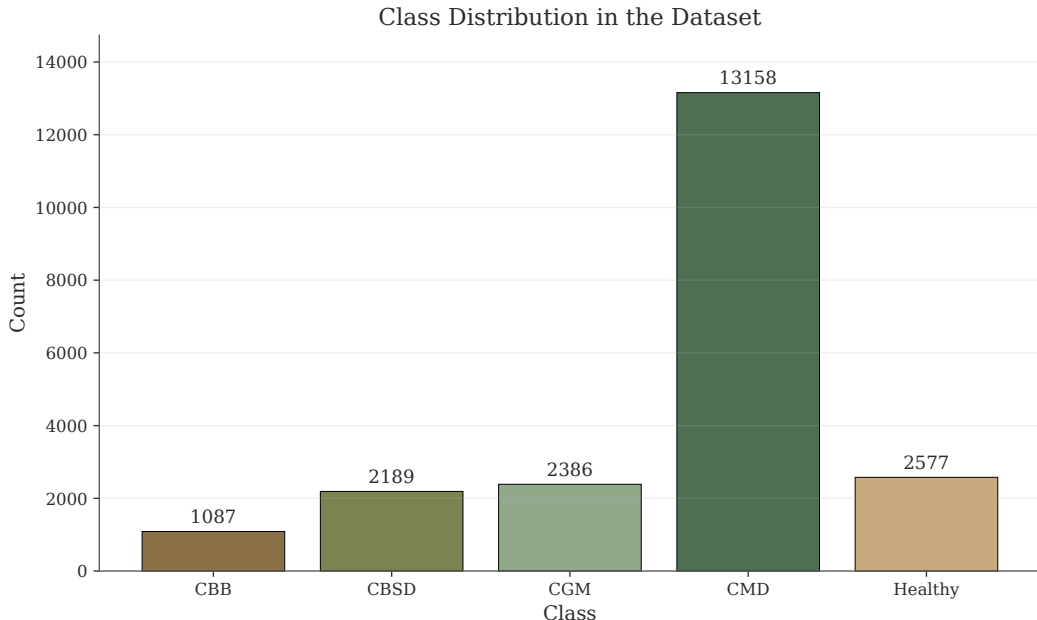


Figure 1: Class distribution in the labeled cassava dataset. The distribution is imbalanced, with Cassava Mosaic Disease (CMD) representing the dominant class. This imbalance motivated the use of validation macro F1-score as the primary selection metric and the inclusion of an imbalance-aware loss condition in Experiment 2.

The validated dataset was then divided into fixed train, validation, and test splits using a stratified 70/15/15 split with a fixed random seed. Stratification was used so that the class proportions remained approximately consistent across the three partitions. The resulting split sizes were 14,977 training images, 3,210 validation images, and 3,210 test images, as summarized in Table 1. These splits were saved and reused throughout the study. Keeping the split fixed was essential to the controlled design because differences between models or training strategies should reflect the experimental variable being tested, not changes in the data partition.

Table 1: Dataset and split summary. The labeled dataset was divided into fixed stratified train, validation, and test splits and reused across all experiments.

Split	Count	Percentage	Purpose
Train	14,977	70.0%	Model fitting
Validation	3,210	15.0%	Model and setup selection
Test	3,210	15.0%	Final held-out evaluation
Total	21,397	100.0%	Full labeled dataset

3.3 Base Preprocessing and Data Pipelines

A shared base preprocessing pipeline was defined before running the modeling experiments. The final input size was set to 224×224 pixels, the resize policy was direct square resizing, and normalization was performed using dataset-specific statistics computed from the training split only. The resulting training-split channel means were [0.4312, 0.4977, 0.3146], and the training-split channel standard deviations were [0.2203, 0.2234, 0.2116].

Computing normalization statistics only on the training split avoided information leakage from the validation and test sets. This decision also kept preprocessing consistent with the study’s reproducibility goal, meaning every model saw images transformed through the same base pipeline unless a later experiment intentionally introduced a training-time augmentation difference.

The choice of direct square resizing to 224×224 was informed by the exploratory image-size analysis. Since all raw images had the same 800×600 shape, preprocessing did not need to account for heterogeneous image dimensions. The main practical decision was therefore how to convert a uniform 4:3 image into a fixed model input size. The 224×224 size was selected to keep the models lightweight and computationally manageable while remaining compatible with standard CNN image-classification pipelines. The study also compared candidate preprocessing policies conceptually, but the final locked base preprocessing used direct resizing followed by tensor conversion and train-split normalization.

A key design distinction in the study is the difference between base preprocessing and data augmentation. Base preprocessing refers to the deterministic image preparation shared across the study, which consisted of resizing, converting to a tensor, and normalizing. Data augmentation refers to stochastic training-time transformations used as an experimental variable in Experiment 2. This distinction matters because Experiment 1 was designed to isolate architecture, so its train and validation pipelines used the same minimal preprocessing without advanced augmentation. In contrast, Experiment 2 intentionally varied the training pipeline to test whether augmentation and imbalance-aware training improved performance.

For Experiment 1, both the training and validation transforms used the shared base preprocessing pipeline. This kept the architecture ablation clean because the baseline CNN, residual CNN, and residual attention CNN were trained under the same preprocessing and evaluation conditions. For Experiment 2, the validation transform remained clean and fixed, while the training transform changed according to the setup being tested. Setup A used basic augmentation through random horizontal flipping. Setup B added stronger field-oriented augmentation, including random horizontal flipping, rotation, color jitter, and Gaussian blur. Setup C used the same augmentation pipeline as Setup B, with imbalance-aware loss added later during training. Representative visual examples of these preprocessing and augmentation choices are shown in Figure 2.

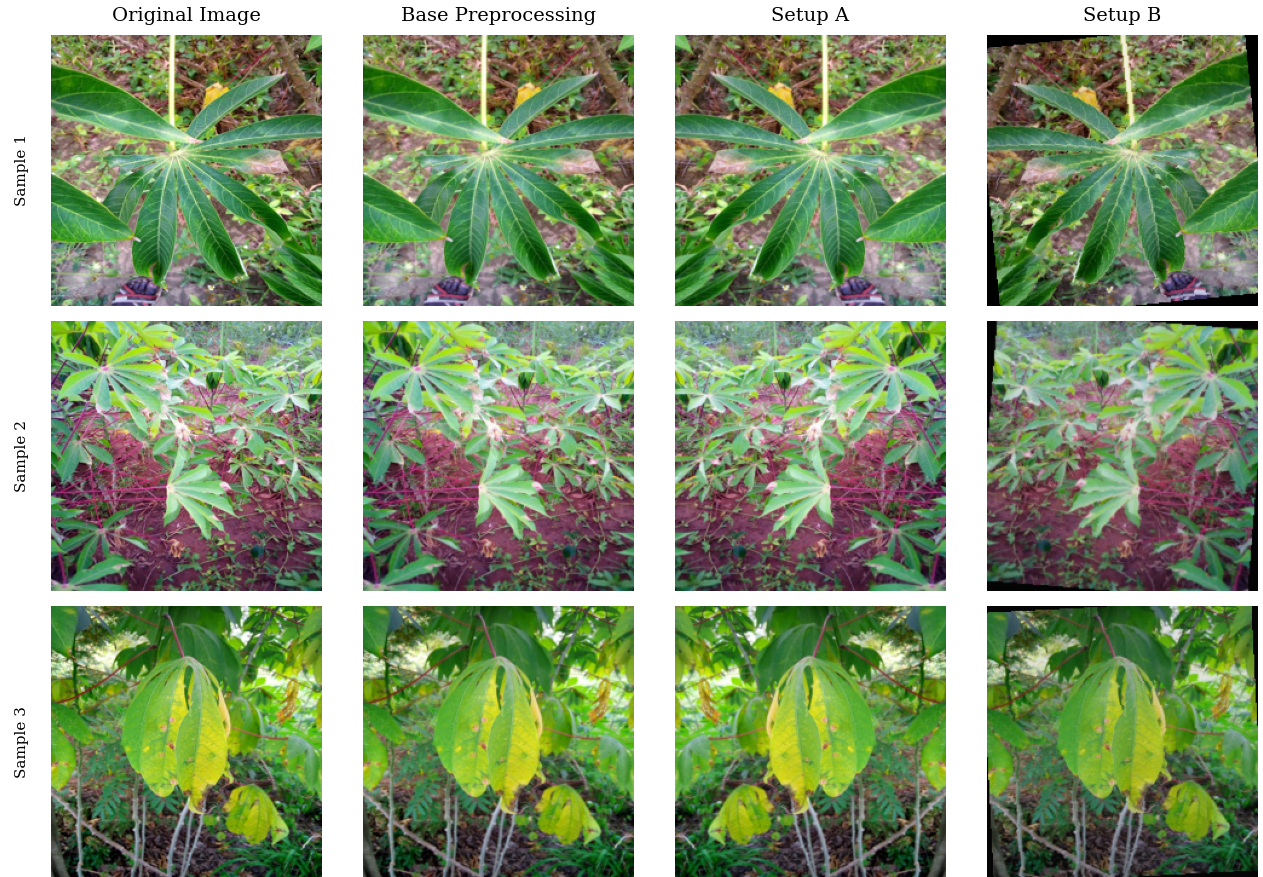


Figure 2: Representative examples of preprocessing and augmentation applied to validation images. The base preprocessing uses resizing and train-split normalization. Setup A adds basic horizontal flipping, while Setup B applies stronger field-oriented transformations including rotation, color jitter, and blur. Setup C uses the same visual augmentation pipeline as Setup B, with weighted cross-entropy added during training.

The final data pipeline therefore supported both reproducibility and controlled experimentation. The deterministic base pipeline ensured that all models were evaluated on consistently processed images, while the experiment-specific training pipelines allowed the second experiment to test whether additional augmentation or class-imbalance handling improved generalization.

3.4 Model Family and Architecture Design

The model family was designed as a controlled progression of lightweight convolutional classifiers. Rather than comparing unrelated architectures, the study used three related CNN variants: a baseline CNN, a residual CNN, and a residual CNN with lightweight channel attention. This design made the architecture comparison more interpretable because each model introduced a specific modification over the previous one. The baseline CNN served as the reference point, the residual CNN tested the effect of residual learning, and the residual attention CNN tested whether channel-wise feature recalibration provided additional value beyond residual connections alone. Residual connections were motivated by prior work on residual learning [5], while the attention component was motivated by channel-attention and convolutional attention mechanisms [6, 13]. The architecture progression is summarized in Figure 3.

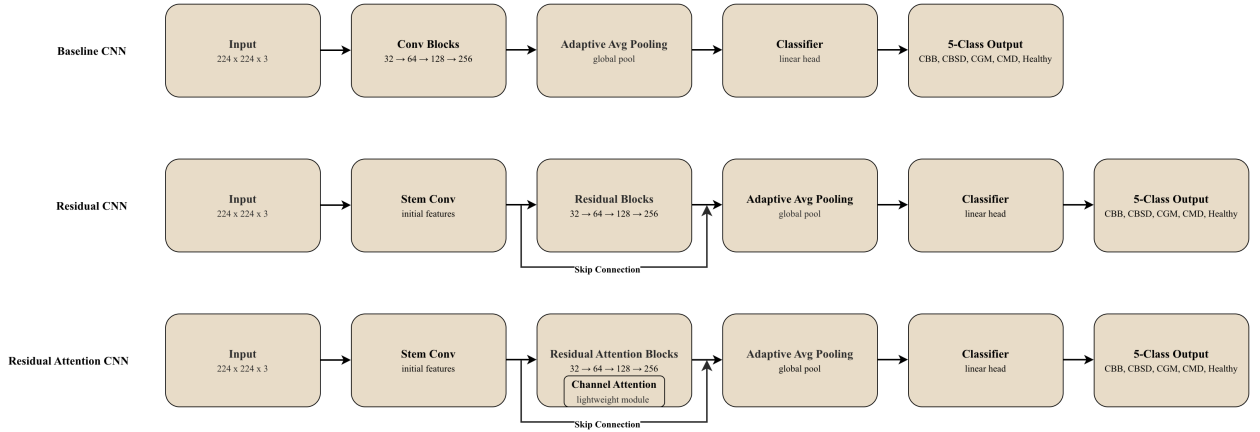


Figure 3: Architecture progression used in the controlled architecture ablation. The baseline CNN provides a plain convolutional reference model, the residual CNN introduces skip connections, and the residual attention CNN adds lightweight channel attention on top of the residual design.

3.4.1 Baseline CNN

The baseline model was a plain convolutional neural network that used a sequence of convolutional feature-extraction stages followed by a linear classifier. Each convolutional block consisted of a 3×3 convolution, batch normalization, and a ReLU activation. The convolution used padding of one pixel so that spatial resolution was preserved within each convolutional block, while downsampling was handled separately using max pooling. The baseline model used four feature stages with channel widths of 32, 64, 128, and 256. After the final convolutional stage, adaptive average pooling reduced the spatial feature map to a 1×1 representation, which was flattened and passed to a linear classifier with five output units, one for each cassava class.

In this model, the architecture was intentionally simple. It did not include residual skip connections or attention modules. Its role was to provide a clean reference point for the architecture ablation. This

means that if later models improved over this baseline, those gains could be interpreted relative to a straightforward CNN feature extractor rather than an already complex architecture.

3.4.2 Residual CNN

The second model modified the baseline idea by replacing the plain convolutional stages with residual blocks. Each residual block used two 3×3 convolutional layers, each followed by batch normalization, with a ReLU activation after the first convolution and again after adding the shortcut connection. The shortcut path was an identity mapping when the input and output dimensions matched. When the number of channels or stride required a dimensional change, the shortcut used a 1×1 convolution followed by batch normalization to project the input to the correct shape before addition.

The residual CNN used an initial convolutional stem mapping the input image from 3 channels to 32 channels, followed by four residual stages with output widths of 32, 64, 128, and 256 channels. Max pooling was used after the first three residual stages for downsampling, and adaptive average pooling was used after the final stage. As in the baseline model, the pooled representation was flattened and passed to a linear classifier with five output units.

The purpose of this architecture was to test whether residual learning improved performance in the cassava field-image setting. Conceptually, residual connections allow a block to learn a modification to its input rather than an entirely new transformation from scratch. In this study, the residual CNN therefore represented a moderate architectural increase over the baseline while still remaining lightweight and reproducible.

3.4.3 Residual Attention CNN

The third model extended the residual CNN by adding lightweight channel attention inside each residual block. The channel-attention module first applied adaptive average pooling to summarize each channel into a 1×1 descriptor. This descriptor was then passed through a small two-layer 1×1 convolutional multilayer perceptron with a ReLU activation and a final sigmoid activation. The resulting channel weights were multiplied back into the feature map, allowing the network to emphasize or suppress feature channels before the residual addition. The reduction ratio was set to 16, with a minimum reduced channel width of 8.

The residual attention CNN kept the same overall stage structure as the residual CNN: an initial convolutional stem, four residual-style stages with channel widths of 32, 64, 128, and 256, adaptive average pooling, and a five-class linear classifier. The main difference was that each residual block became a residual attention block, where channel attention was applied after the second batch-normalized convolution and before the shortcut addition.

This model was included to test whether channel-wise feature recalibration improved the model’s ability to identify disease-relevant visual patterns. Since plant disease images may contain cluttered backgrounds, varying lighting, and visually subtle symptoms, channel attention was a reasonable lightweight mechanism to test. Importantly, the model did not introduce a completely different architecture family. It instead added attention on top of the residual CNN, thereby preserving the controlled progression of the ablation.

3.4.4 Controlled Architecture Progression

The three models were intentionally structured as an incremental architecture family. The baseline CNN tested the performance of a simple convolutional classifier. The residual CNN then added skip connections to test the value of residual learning. The residual attention CNN then added channel attention to test whether feature recalibration provided an additional benefit beyond residual learning. This progression allowed the architecture experiment to ask two clean questions. The first was whether residual connections improved over a plain CNN, and the second was whether channel attention improved over residual connections alone.

All three architectures produced logits of shape $(N, 5)$, where N is the batch size and 5 is the number of cassava classes. The models were also checked with forward-pass sanity tests before training. Trainable parameter counts were computed for each model and later used as part of the performance-complexity analysis. The baseline CNN had the fewest parameters, while the residual and residual attention models introduced additional capacity through residual blocks and attention modules. Table 2 summarizes the three architectures.

Table 2: Architecture summary for the three CNN variants used in Experiment 1. Each model outputs logits for the same five cassava classes.

Model	Key Components	Residual	Attention	Parameters
Baseline CNN	Four plain convolutional feature stages with channel widths 32, 64, 128, and 256, followed by adaptive average pooling and a linear classifier.	No	No	390,181
Residual CNN	Initial convolutional stem followed by four residual stages with channel widths 32, 64, 128, and 256.	Yes	No	1,227,685
Residual Attention CNN	Residual CNN structure with lightweight channel-attention modules inside the residual blocks.	Yes	Yes	1,239,981

3.5 Training Protocol

All experiments used a shared training and evaluation framework so that model comparisons would be controlled and reproducible. The same dataset splits, preprocessing decisions, batch size, optimizer family, scheduler type, number of epochs, and evaluation metrics were used unless a specific experiment intentionally changed one component. This was important because the purpose of the study was not only to obtain high accuracy, but to understand which design choices were responsible for differences in performance.

Training was performed using the saved artifacts from the data validation stage. The Kaggle dataset was re-downloaded into the Colab runtime, while the fixed train, validation, and test split files, label mapping, class-count summary, preprocessing configuration, and pipeline decisions were loaded from Google Drive. Image paths were then rebuilt against the runtime image directory, and the saved preprocessing settings were recovered. This ensured that the modeling and experiments reused the exact splits and preprocessing choices established during the data preparation stage.

The global modeling defaults were fixed before the main experiments. The batch size was set to 32,

the number of training epochs was set to 30, the learning rate was 10^{-3} , and the weight decay was 10^{-4} . The optimizer was AdamW, and the learning-rate scheduler was ReduceLROnPlateau with validation macro F1 as the quantity being monitored. The scheduler used mode “max”, a reduction factor of 0.5, and patience of 2 epochs. Cross-entropy loss was used for Experiment 1 and for the non-imbalance-aware training setups in Experiment 2.

The training loop followed the same structure for all model variants. During each epoch, the model was trained on the training loader, evaluated on the validation loader, and the resulting training and validation loss, accuracy, and macro F1-score were recorded. The best checkpoint was selected using validation macro F1. If two epochs tied in validation macro F1, validation accuracy was used as a secondary tie-breaker. This selection rule reflected the study’s emphasis on class-sensitive performance while still retaining accuracy as a useful supporting measure.

The test set was held out from model selection. It was used only after the best model or setup had been selected using the validation results. This separation was important because the validation set guided architecture and training-strategy selection, while the test set provided a final held-out estimate of performance for the selected model. The same evaluation function was used for validation and test evaluation, ensuring that metrics were computed consistently across all stages of the study.

In Experiment 2, the loss function differed only for the imbalance-aware setup. Setup A and Setup B used standard cross-entropy loss, while Setup C used weighted cross-entropy. The class weights were computed from the training split, reflecting the observed class imbalance in the data. The motivation for this weighted-loss condition is discussed in the Experiments section, while the Methods section records it as a key distinction in the training protocol.

3.6 Evaluation Metrics

The study used both aggregate and class-sensitive metrics to evaluate model behavior. The main metrics were macro F1-score, accuracy, per-class recall, confusion matrices, and trainable parameter count. Macro F1-score computed the F1-score separately for each class and then averaged across classes, giving equal weight to each disease or health category. Accuracy measured the overall fraction of correctly classified images. Per-class recall measured how often each true class was correctly identified. Confusion matrices were used to inspect which classes were being confused with one another. Trainable parameter count was used as a lightweight efficiency measure.

Validation macro F1-score was the primary model-selection metric. This choice was motivated by the class imbalance observed during exploratory analysis. Elaborating further, if accuracy alone were used, a model could appear strong by performing well on the majority class while still underperforming on minority disease classes. Macro F1-score reduced this risk by giving each class equal influence in the model-selection criterion. Accuracy was still reported because it remains an intuitive measure of overall correctness, but it was treated as a secondary metric rather than the primary selection criterion.

Per-class recall and confusion matrices were included because they provide information that aggregate metrics can hide. In a crop-disease classification setting, the specific type of error matters because confusing one disease for another or failing to detect a minority class may have different practical implications than simply lowering overall accuracy. These class-level metrics therefore helped connect the numerical results to the study’s broader goal of evaluating whether lightweight models behave reliably across all cassava categories.

Parameter count was included because the study was explicitly concerned with lightweight and reproducible model design. A model with slightly higher predictive performance may not be preferable

if it requires substantially more complexity. For that reason, the architecture comparison considered not only validation macro F1 and accuracy, but also the number of trainable parameters required to obtain those results. The goal was to evaluate the tradeoff between performance and model complexity rather than selecting a model solely by predictive metrics.

4. Experiments

The study was organized around two sequential controlled experiments. The first experiment tested architectural design choices, while the second tested training-strategy choices using the strongest architecture from the first experiment. This sequence was intentional. Selecting the architecture first made it possible to evaluate augmentation and imbalance-aware training without mixing those effects with changes in model structure. Across both experiments, the goal was not only to identify the highest-performing configuration, but also to isolate which design choices produced measurable benefits under a fixed experimental protocol.

4.1 Overview of Experimental Design

The experimental design followed a two-stage structure. Experiment 1 was an architecture ablation comparing three related CNN models, namely a baseline CNN, a residual CNN, and a residual CNN with channel attention. This experiment asked whether incremental architectural changes improved cassava classification performance under a fixed training and preprocessing setup. Experiment 2 then fixed the best-performing architecture from Experiment 1 and compared three training strategies: basic augmentation, stronger field-oriented augmentation, and stronger augmentation combined with weighted cross-entropy.

This ordering was used to isolate one design question at a time. If architecture and training strategy had been varied simultaneously, it would have been difficult to determine whether a performance difference came from the model structure, the augmentation pipeline, the loss function, or some interaction among them. By first selecting a model architecture and then testing training strategies using that fixed architecture, the study produced cleaner comparisons and more interpretable conclusions.

The experiments also shared the same general evaluation protocol. All comparisons used the fixed train, validation, and test splits detailed in the Methods section. Validation macro F1-score was used as the primary selection metric, with accuracy, per-class recall, confusion matrices, and parameter count used as supporting metrics. Final test evaluation was reserved for selected models or configurations after validation-based selection.

4.2 Experiment 1: Architecture Ablation

Experiment 1 was designed as an architecture-only ablation. The purpose was to test whether modest architectural additions improved the performance of a lightweight CNN classifier for cassava leaf disease classification. The three compared models were the baseline CNN, the residual CNN, and the residual attention CNN. These models formed an incremental sequence in which the baseline CNN provided a plain convolutional reference model, the residual CNN added skip connections, and the residual attention CNN added channel attention on top of the residual design.

To keep the comparison interpretable, the training and evaluation setup was held fixed across all three architectures. Each model used the same train, validation, and test split; the same base preprocessing

pipeline; the same input size; the same optimizer; the same scheduler; the same batch size; the same number of epochs; the same cross-entropy loss; and the same training and evaluation loop. No advanced augmentation or imbalance-aware loss was introduced in this experiment. This was important because the purpose of Experiment 1 was to isolate architectural effects. If augmentation or loss weighting had been introduced at this stage, the comparison would no longer cleanly answer whether the architectural changes themselves were responsible for performance differences.

The main question in Experiment 1 was whether the added complexity of residual learning and channel attention produced enough improvement to justify their use in a lightweight field-image classifier. The baseline CNN represented the simplest model in the architecture family. The residual CNN tested whether skip connections improved representation learning and optimization. The residual attention CNN tested whether channel-wise feature recalibration provided additional benefit beyond residual connections alone. The models were selected and ranked primarily by validation macro F1-score, while accuracy and parameter count were used to interpret the performance-complexity tradeoff.

The expected value of this experiment was not only to identify the best architecture, but also to determine whether the progression from baseline to residual to residual attention produced a meaningful empirical pattern. A small gain from a much larger model would be less compelling for the study’s lightweight deployment-oriented goal. In contrast, a meaningful improvement from a modest architectural addition would support the use of targeted architecture refinement in this setting.

4.3 Experiment 2: Training-Strategy Ablation

Experiment 2 was designed to test whether training interventions improved performance after the architecture had been selected. The best-performing architecture from Experiment 1 was fixed and retrained from scratch under three training setups. This design allowed the experiment to focus on the effect of training strategy rather than model structure. In all three setups, the architecture, fixed data splits, validation and test transforms, optimizer, scheduler, batch size, number of epochs, training loop, and validation-based selection rule remained constant. The only intended differences were the training-time augmentation pipeline and, for Setup C, the loss function.

Setup A used basic augmentation. Its training transform consisted of the shared resizing and normalization pipeline plus random horizontal flipping. This setup served as a simple augmentation baseline. It tested whether a modest and common image augmentation was sufficient to improve generalization without adding substantial training complexity.

Setup B used stronger field-oriented augmentation. In addition to random horizontal flipping, it included random rotation, color jitter, and Gaussian blur. These transformations were chosen because field images can vary in orientation, lighting, color appearance, and image sharpness. The purpose of Setup B was to test whether a stronger augmentation pipeline better simulated real field-image variability and improved validation performance compared with the simpler Setup A.

Setup C used the same field-oriented augmentation pipeline as Setup B, but added weighted cross-entropy loss. The motivation for this setup came from the class imbalance observed during data exploration, especially the dominance of CMD in the dataset. The weighted loss was computed from the training split class counts so that minority classes received greater relative weight during optimization. This setup tested whether imbalance-aware training improved class-sensitive performance beyond augmentation alone.

Together, the three training setups formed a controlled training-strategy ablation. Setup A tested whether simple augmentation was enough. Setup B tested whether stronger field-oriented transfor-

mations improved generalization. Setup C tested whether adding class-imbalance handling further improved macro-level and class-level performance. Because the selected architecture and all other training settings were held fixed, differences among the setups could be interpreted primarily as effects of the training strategy.

4.4 Experimental Questions

The two experiments were designed to answer a set of connected design questions:

1. Does residual learning improve performance over a plain CNN baseline?
2. Does channel attention improve performance beyond residual learning alone?
3. Do stronger field-oriented augmentations improve generalization compared with basic augmentation?
4. Does weighted cross-entropy improve class-sensitive behavior under class imbalance?
5. Which design choices provide the best balance among predictive performance, model complexity, reproducibility, and practical deployment relevance?

These questions connect the experimental design back to the central goal of the study. The purpose was not simply to report the best numerical result, but to understand which lightweight architectural and training choices were actually useful for cassava disease classification on realistic field images.

5. Results and Discussion

This section presents and interprets the results from the two controlled experiments. The results are organized around the same structure as the experimental design. First, the architecture ablation is evaluated; second, the training-strategy ablation is evaluated using the selected architecture from Experiment 1; and finally, the selected final model is analyzed on the held-out test set. The discussion emphasizes not only which model performed best, but also what the results suggest about the tradeoff between predictive performance, model complexity, reproducibility, and practical deployment relevance.

5.1 Experiment 1: Architecture Ablation Results

Experiment 1 compared the baseline CNN, residual CNN, and residual attention CNN under the same preprocessing, training, and evaluation conditions. The residual attention CNN achieved the strongest validation performance, with a best validation macro F1-score of 0.7260 and validation accuracy of 0.8427 at epoch 20. The residual CNN ranked second, with validation macro F1-score of 0.7107 and validation accuracy of 0.8399 at epoch 30. The baseline CNN ranked third, with validation macro F1-score of 0.6776 and validation accuracy of 0.8234 at epoch 28. The parameter counts were 390,181 for the baseline CNN, 1,227,685 for the residual CNN, and 1,239,981 for the residual attention CNN. These results are summarized in Table 3.

These results show a clear performance improvement as the architecture becomes more structured. Moving from the baseline CNN to the residual CNN increased validation macro F1 from 0.6776 to 0.7107, suggesting that residual connections improved the model’s ability to learn useful image

Table 3: Experiment 1 architecture ablation results. Validation macro F1 was used as the primary model-selection metric.

Model	Parameters	Best Epoch	Validation Accuracy	Validation Macro F1
Baseline CNN	390,181	28	0.8234	0.6776
Residual CNN	1,227,685	30	0.8399	0.7107
Residual Attention CNN	1,239,981	20	0.8427	0.7260

representations. Adding channel attention increased validation macro F1 further to 0.7260, while only slightly increasing the parameter count relative to the residual CNN. This is important because the study is not simply concerned with maximizing model size; it is concerned with whether added complexity produces enough benefit to justify itself.

The complexity-performance comparison makes this tradeoff clearer. Moving from the baseline CNN to the residual CNN added 837,504 parameters, a 214.6% increase, and improved validation macro F1 by 0.0331, or about 4.89% relative to the baseline. Moving from the residual CNN to the residual attention CNN added only 12,296 parameters, a 1.00% increase, while improving validation macro F1 by 0.0153, or about 2.15% relative to the residual CNN. This means the attention module produced a smaller absolute improvement than the full residual redesign, but it did so with much less added parameter cost. Measured as macro-F1 gain per 10,000 added parameters, the residual attention addition was substantially more parameter-efficient than the jump from the baseline CNN to the residual CNN.

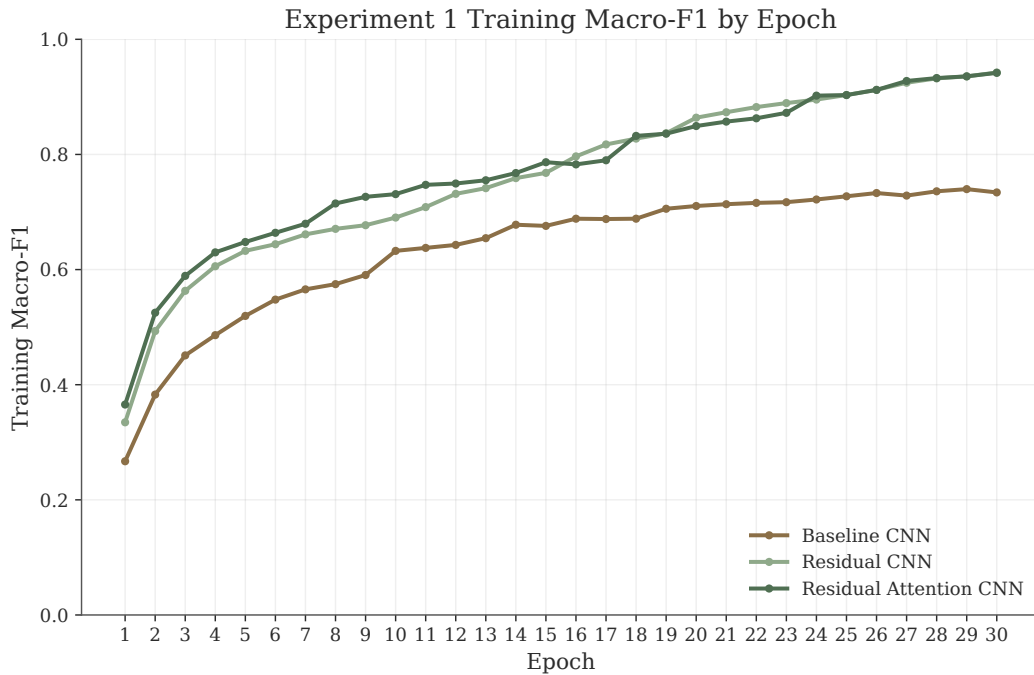


Figure 4: Experiment 1 training macro-F1 curves for the baseline CNN, residual CNN, and residual attention CNN. The residual-style architectures reached higher training macro F1 than the baseline CNN, while final model selection was still based on validation macro F1.

The training macro-F1 curves in Figure 4 provide additional context for the architecture comparison. The baseline CNN improved steadily but remained below the two residual models by the end of training.

The residual CNN and residual attention CNN reached substantially higher training macro F1-scores, indicating that the residual-style architectures had greater learning capacity. However, the validation results are more important for model selection than the training curves. The residual attention CNN was not selected based on training behavior alone; it also achieved the best validation macro F1, which supports selecting it as the strongest architecture in the controlled comparison.

5.2 Experiment 1 Discussion

The Experiment 1 results support the value of targeted architectural refinement. The baseline CNN provided a useful reference point, but its lower macro F1-score suggests that the plain convolutional architecture was less effective for the field-image cassava task. The residual CNN improved performance, likely because residual blocks made feature learning easier and allowed information to flow through the network more effectively. The residual attention CNN then produced the best validation result, supporting the idea that channel-wise feature recalibration added useful information beyond residual learning alone.

The parameter-count comparison is also important. The residual CNN required substantially more parameters than the baseline CNN, so its improvement must be interpreted as a performance-complexity tradeoff. However, the residual attention CNN added only a small number of parameters beyond the residual CNN while producing the best validation macro F1. This makes the attention addition more defensible than if it had required a much larger increase in model size. In other words, the result suggests that channel attention was a relatively efficient architectural addition within this model family.

The architecture ablation also shows why controlled comparisons are useful. If the study had only trained the final residual attention model, it would not be clear whether its performance came from residual learning, attention, or both. Because the models were compared sequentially, the result can be interpreted as evidence for both residual learning and channel attention within this specific model family. This supports the broader claim that modest, targeted architectural choices can improve lightweight cassava leaf disease classification under realistic field-image conditions.

5.3 Experiment 2: Training-Strategy Ablation Results

Experiment 2 fixed the winning architecture from Experiment 1, the residual attention CNN, and retrained it from scratch under three training setups. Setup A used basic augmentation, Setup B used stronger field-oriented augmentation, and Setup C used the same stronger augmentation as Setup B with weighted cross-entropy. Setup A achieved the best validation result, with validation macro F1-score of 0.7449 and validation accuracy of 0.8617 at epoch 22. Setup B ranked second, with validation macro F1-score of 0.7345 and validation accuracy of 0.8564 at epoch 30. Setup C ranked third by validation macro F1, with validation macro F1-score of 0.7333 and validation accuracy of 0.8349 at epoch 24. These results are summarized in Table 4.

The results show that the simplest training strategy performed best. Setup A, which used random horizontal flipping in addition to the shared base preprocessing, outperformed both the stronger augmentation pipeline and the imbalance-aware weighted-loss setup. This does not mean that stronger augmentation or weighted loss is generally ineffective. Rather, in this particular controlled setting, they did not improve validation macro F1 beyond the simpler augmentation strategy.

The comparison among the training strategies also shows that additional training complexity did not translate into better validation macro F1. Setup B had a validation macro F1 that was 0.0104 lower

Table 4: Experiment 2 training-strategy ablation results using the residual attention CNN. Validation macro F1 was used as the primary setup-selection metric.

Setup	Augmentation Strategy	Loss Type	Best Epoch	Val. Acc.	Val. Macro F1
Setup A	Basic augmentation with random horizontal flipping.	Cross-entropy	22	0.8617	0.7449
Setup B	Field-oriented augmentation with horizontal flipping, rotation, color jitter, and Gaussian blur.	Cross-entropy	30	0.8564	0.7345
Setup C	Same augmentation as Setup B.	Weighted cross-entropy	24	0.8349	0.7333

than Setup A, a relative decrease of about 1.40%. Setup C was 0.0116 lower than Setup A, a relative decrease of about 1.56%. Compared with Setup B, Setup C was lower by 0.0012, or about 0.160%. These differences are not extremely large, but they consistently favor the simpler Setup A over the more complex augmentation and weighted-loss strategies.

These results should be interpreted as specific to the fixed split and augmentation choices tested here, rather than as evidence against field-oriented augmentation or imbalance-aware loss in general.

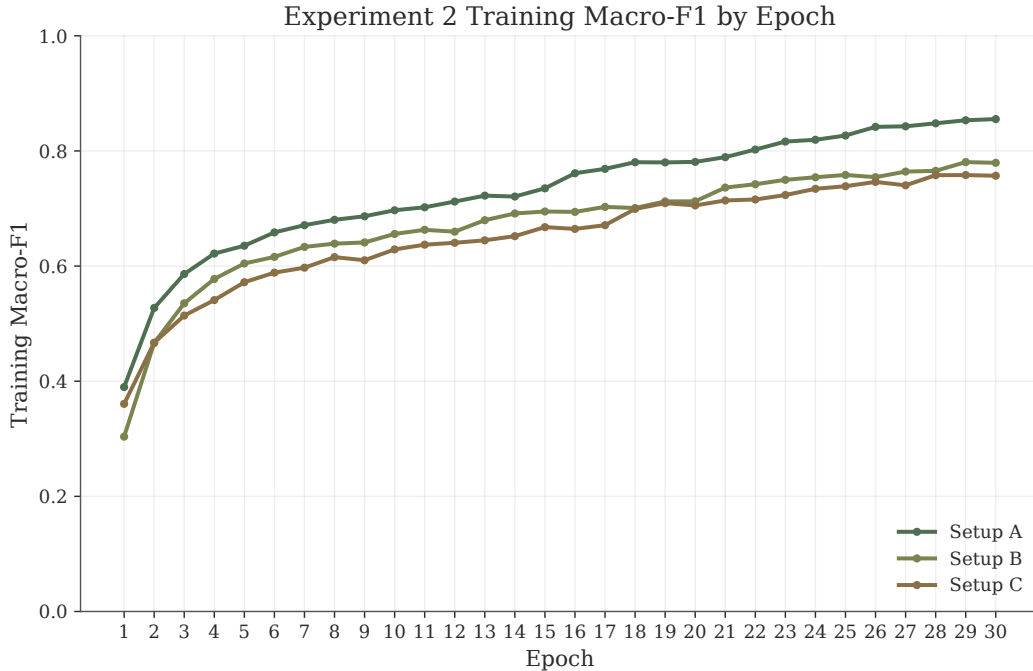


Figure 5: Experiment 2 training macro-F1 curves for the three training strategies using the residual attention CNN. Setup A produced the strongest validation result despite using the simplest augmentation strategy.

The training macro-F1 curves in Figure 5 help explain the difference among the setups. Setup A learned strongly while maintaining the best validation macro F1. Setup B trained more slowly and required

more computation per epoch because of the stronger augmentation operations. Setup C introduced an additional optimization change through weighted cross-entropy, which was justified by the observed class imbalance, but it did not produce the best validation result. The fact that Setup C achieved a validation macro F1 close to Setup B suggests that imbalance-aware loss did not fail entirely, but it did not provide enough improvement to justify replacing Setup A as the final training strategy.

5.4 Final Test Performance

After Experiment 2, the final selected configuration was the residual attention CNN trained under Setup A. This configuration was selected because it achieved the best validation macro F1-score in the training-strategy ablation. On the held-out test set, the final model achieved a test macro F1-score of 0.7365, test accuracy of 0.8589, and test loss of 0.4183. The model retained the same parameter count as the residual attention CNN, 1,239,981 trainable parameters. The final test results are summarized in Table 5.

Table 5: Held-out test performance for the residual attention CNN under the base Experiment 1 preprocessing configuration and the selected Setup A training strategy. Setup A was the final selected configuration.

Architecture	Training Setup	Test Accuracy	Test Macro F1	Test Loss	Parameters
Residual Attention CNN	Base	0.8452	0.7317	0.4918	1,239,981
Residual Attention CNN	Setup A	0.8589	0.7365	0.4183	1,239,981

The final test results are consistent with the validation-based selection. Relative to the architecture-only evaluation of the same residual attention model, Setup A produced a modest improvement in final held-out performance. The improvement was not dramatic, but it was achieved without changing the architecture or adding a more complex loss function. The test macro F1 was slightly lower than the best validation macro F1, which is expected when moving from validation-based selection to a held-out test set and suggests that the validation estimate was broadly, but not perfectly, predictive of final performance.

The final test performance supports the main empirical pattern of the study. The strongest configuration did not come from the most complicated training setup. Instead, the best result came from combining the strongest lightweight architecture with a simple augmentation strategy. This is an important result because it shows that added training complexity should be evaluated rather than preemptively assumed to be beneficial.

5.5 Class-Level Behavior

The final selected model showed strong performance on several classes but did not perform equally across all disease categories. The highest test recall was for CMD, with recall of 0.9696. The model also performed reasonably well on CBD and CGM, with recalls of 0.7295 and 0.7318, respectively. Healthy leaves had recall of 0.6684, while CBB had the lowest recall at 0.5092. These class-level recall values are shown in Figure 7.

The confusion matrix in Figure 6 provides a more detailed view of the model’s error pattern. For CMD, the model correctly classified 1,914 test images and made relatively few errors compared with the size of that class. This high recall is consistent with the strong diagonal entry for CMD in the

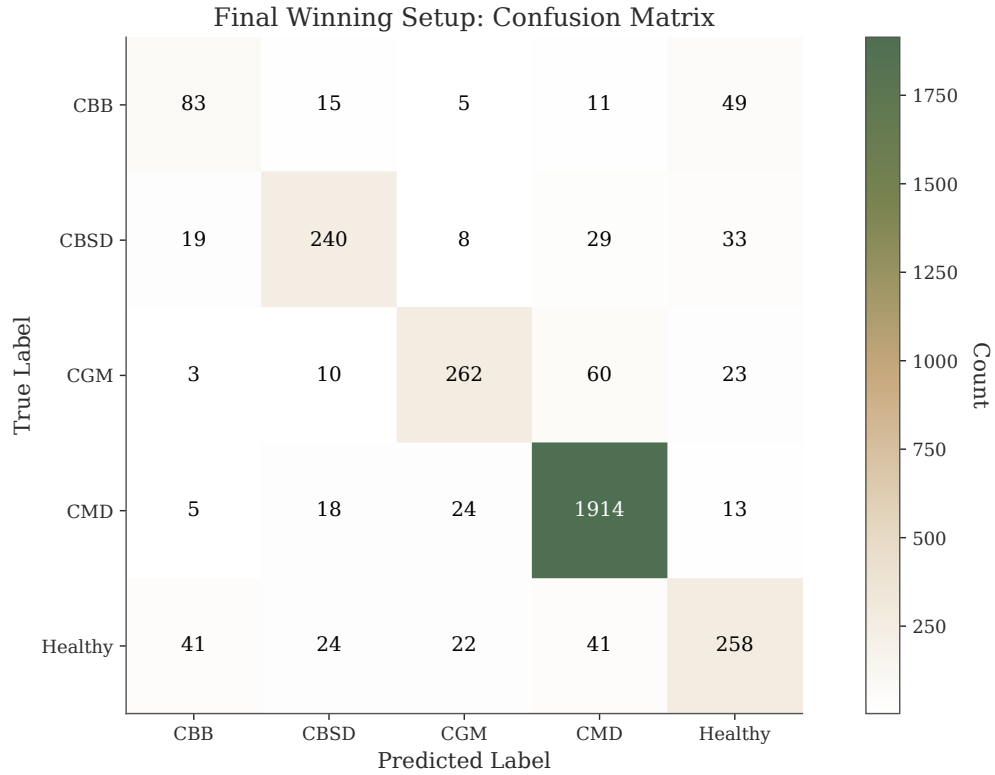


Figure 6: Final test confusion matrix for the selected residual attention CNN trained with Setup A. The model performed strongest on CMD but showed notable confusion between CBB and Healthy leaves.

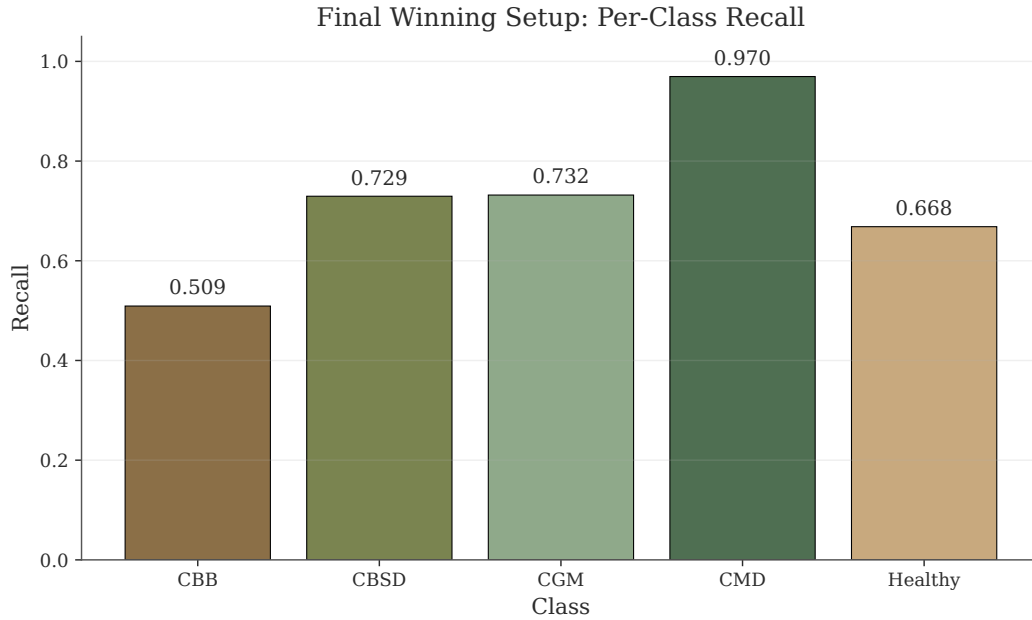


Figure 7: Per-class recall for the final selected model on the held-out test set. Recall was highest for CMD and lowest for CBB, showing that the final model did not perform uniformly across disease categories.

confusion matrix. However, the model had more difficulty with CBB and Healthy images. For true CBB images, 83 were correctly classified, but 49 were predicted as Healthy. For true Healthy images, 258 were correctly classified, but 41 were predicted as CBB and another 41 were predicted as CMD. These patterns suggest that the model sometimes confused disease and healthy categories, especially for CBB and Healthy.

These class-level results confirm the need for the class-sensitive metrics discussed in Section 3.6, since overall accuracy alone would hide uneven behavior across classes. Simply put, a model can perform very well on the dominant CMD class while still struggling with smaller or visually ambiguous classes. Since the intended application is disease recognition, class-specific behavior matters because errors on minority disease classes may be more practically important than a small change in aggregate accuracy.

The comparison between the final model and the Experiment 1 test confusion matrix also suggests that the final Setup A training strategy changed the class-level behavior. The final model improved CMD recall substantially and improved CGM recall slightly, but CBB recall decreased compared with the Experiment 1 test result. This tradeoff shows that a higher aggregate test accuracy or macro F1-score does not mean every class improves uniformly. The final model is therefore best understood as the strongest overall configuration under the chosen selection rule, not as a model that solved every class equally well.

5.6 Overall Discussion and Synthesis

Across both experiments, the strongest pattern is that targeted architecture changes were more clearly beneficial than the additional training interventions tested here. Experiment 1 showed that the residual attention CNN outperformed both the plain baseline and the residual-only model, supporting the value of residual learning and lightweight channel attention. Experiment 2 then showed that, once the best architecture was fixed, the simplest augmentation strategy performed better than stronger augmentation and weighted cross-entropy.

This outcome is useful because it challenges a common assumption that more complex training interventions automatically improve model performance. Setup B was designed to reflect field-image variability more aggressively, and Setup C was justified by the class imbalance observed during exploratory analysis. Both were reasonable experimental choices. However, the results show that reasonable design motivations do not guarantee empirical improvement. In this dataset and training setup, stronger augmentation may have introduced transformations that were not as helpful for the validation distribution, and weighted cross-entropy may have changed the optimization objective without improving the final class-sensitive metric enough to win the comparison.

The final model therefore reflects a balanced design choice, combining a residual attention CNN with simple augmentation. This configuration achieved the best validation-selected training strategy and the strongest final test performance among the evaluated setups. The result supports the broader goal of the study, which is that lightweight cassava disease classification can benefit from carefully chosen architectural improvements, but the value of additional training complexity must be tested directly. Controlled ablations were essential because they separated architecture effects from training-strategy effects and made the final conclusion more interpretable.

6. Conclusion

6.1 Main Takeaway

This study evaluated lightweight CNN-based models for cassava leaf disease classification on realistic field images. The goal was not to build the largest possible classifier, but to run a controlled study of which architectural and training choices improved performance while preserving reproducibility and practical deployment relevance. Across the two experiments, the residual attention CNN was the strongest architecture, and the simple augmentation strategy in Setup A was the strongest training strategy. The main takeaway is that targeted architectural refinement helped more clearly than added training complexity. Residual learning and lightweight channel attention improved validation macro F1, while stronger augmentation and weighted cross-entropy did not outperform the simpler training setup. This supports the broader conclusion that lightweight cassava disease classifiers can benefit from carefully chosen design improvements, but those improvements need to be tested through controlled comparisons rather than assumed in advance.

6.2 Limitations and Unanswered Questions

Several limitations remain. First, the study used a single dataset and did not evaluate the final model on an external cassava dataset or a separate geographic domain. Although the internal train, validation, and test split allowed for a controlled comparison, it does not prove that the same model would generalize across different farms, regions, cameras, seasons, or image-collection conditions. This is especially important because the motivating use case is real field deployment, where image quality and disease presentation may differ from the dataset used in this study.

Second, the study was limited to a small family of custom lightweight CNNs. This was intentional because the study focused on controlled architecture progression rather than comparing every possible model family. However, it leaves open whether other lightweight architectures, pretrained models, or transfer-learning approaches could achieve stronger performance with similar or lower practical cost. The jump from the baseline CNN to the residual CNN also increased parameter count substantially, so an intermediate architecture with fewer than one million parameters could be worth studying to better identify where the performance-complexity tradeoff begins to flatten. Because the experiments were single-run comparisons, small differences such as the gain from residual attention over the residual CNN should also be treated cautiously until repeated-seed experiments confirm their stability.

Third, the final model still showed uneven class-level behavior. The model performed especially well on CMD, but CBB had much lower recall and was often confused with Healthy images. This matters because a field-deployed disease-recognition tool is only as useful as its reliability across the disease classes that matter in practice. In other words, the underlying limitation that the current experiments do not sufficiently explain why CBB is difficult or how best to improve performance for that class remains.

Fourth, the training-strategy experiment leaves some unanswered questions. Setup A outperformed the stronger field-oriented augmentation setup and the weighted-loss setup, but the experiment does not prove exactly why. Stronger augmentation may have introduced transformations that were not well aligned with the validation distribution, or weighted cross-entropy may have shifted the optimization objective without improving the most important class-level errors. Additional ablations would be needed to isolate which individual augmentation operations helped or hurt.

Finally, the study did not include statistical confidence intervals, mobile latency benchmarks, memory-use measurements, or real on-device testing. As a result, the study’s lightweight framing is supported by parameter counts, but not yet by measured inference speed or deployment behavior.

6.3 Future Work

Future work should first test generalization beyond the current dataset. Evaluating the final model on external cassava or plant-disease datasets would show whether the conclusions hold under different field conditions. Domain-shift testing across farms, regions, seasons, lighting conditions, camera types, and image quality levels would be especially valuable because real deployment would likely involve images that differ from the training distribution.

A second direction is to study deployment more directly. Since this study emphasizes lightweight and practical models, future work should measure inference latency, memory usage, and model size on mobile or edge devices. The model could also be tested under realistic mobile-image conditions, such as motion blur, low resolution, poor lighting, or lower-quality phone cameras. These experiments would make the deployment claim stronger than parameter count alone.

A third direction is to improve class-level reliability. The low recall for CBB suggests the need for targeted error analysis and class-specific interventions. Possible approaches include class-specific augmentation, oversampling weaker classes, stronger disease-specific feature analysis, alternative imbalance-aware losses, or cost-sensitive losses that penalize disease false negatives more heavily. The CBB/Healthy confusion is particularly important because misclassifying a diseased plant as healthy could be harmful in practice.

A fourth direction is to broaden the model comparison. This could include pretrained lightweight models, transfer learning, additional custom CNN variants, pruning, quantization, or other compression techniques. These experiments would help determine whether the residual attention CNN is a strong design choice relative to a wider set of efficient alternatives.

Finally, future work could study calibration and uncertainty estimation. A field-deployed decision-support system should not only output a predicted disease class, but also communicate when the model is uncertain. This would be particularly important for visually ambiguous cases or classes where the model has lower recall.

6.4 Ethical and Social Implications

Cassava disease classification has potential social value because earlier and more accessible disease recognition could support crop monitoring, farmer decision-making, and food-security efforts. A lightweight image-based system could be useful in settings where expert diagnosis is difficult to access, especially if it can run on common mobile devices or low-resource hardware.

At the same time, the ethical risks are significant. An incorrect prediction could lead to poor crop-management decisions, delayed treatment, unnecessary intervention, or misplaced confidence in a diseased crop. These risks are particularly serious when the model’s performance is uneven across classes. For example, if a model misses a disease class such as CBB or confuses diseased leaves with Healthy leaves, the system could provide false reassurance to users. This means that aggregate accuracy is not enough for responsible deployment; class-level reliability and uncertainty communication are necessary.

The model should therefore be treated as a decision-support tool rather than an autonomous authority.

It should not replace expert agronomic judgment, especially in high-stakes disease-management decisions. Any deployment should clearly communicate the model’s limitations, provide confidence or uncertainty information when possible, and be tested on representative field images from the intended deployment environment. Data representativeness is also an ethical concern. If the training data do not reflect the full range of regions, devices, farms, disease stages, and environmental conditions where the model is used, performance may be uneven across user communities. Responsible agricultural AI systems should therefore prioritize transparency, validation, and accountability alongside predictive performance.

References

- [1] Quan Huu Cap, Hiroyuki Uga, Satoshi Kagiwada, and Hitoshi Iyatomi. Leafgan: An effective data augmentation method for practical plant disease diagnosis. *arXiv preprint arXiv:2002.10100*, 2020. doi: 10.48550/arXiv.2002.10100. URL <https://arxiv.org/abs/2002.10100>.
- [2] Food and Agriculture Organization of the United Nations. Importance of cassava. <https://www.fao.org/agriculture/crops/thematic-sitemap/theme/pests/cassava-diseases/importance-cassava/en/>. Accessed April 3, 2026.
- [3] Penghui Gui, Wenjie Dang, Feiyu Zhu, and Qijun Zhao. Towards automatic field plant disease recognition. *Computers and Electronics in Agriculture*, 191:106523, 2021. doi: 10.1016/j.compag.2021.106523. URL <https://www.sciencedirect.com/science/article/pii/S0168169921005408>.
- [4] Reem Ibrahim Hasan, Suhaila Mohd Yusuf, and Laith Alzubaidi. Review of the state of the art of deep learning for plant diseases: A broad analysis and discussion. *Plants*, 9(10):1302, 2020. doi: 10.3390/plants9101302. URL <https://www.mdpi.com/2223-7747/9/10/1302>.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. URL https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.
- [6] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Hu_Squeeze-and-Excitation_Networks_CVPR_2018_paper.html.
- [7] Makerere University AI Lab. Cassava leaf disease classification. <https://www.kaggle.com/competitions/cassava-leaf-disease-classification/overview>. Kaggle competition page. Accessed April 3, 2026.
- [8] Ernest Mwebaze, Timnit Gebru, Andrea Frome, Solomon Nsumba, and Jeremy Tusubira. icassava 2019 fine-grained visual categorization challenge. *arXiv preprint arXiv:1908.02900*, 2019. doi: 10.48550/arXiv.1908.02900. URL <https://arxiv.org/abs/1908.02900>.
- [9] Amanda Ramcharan, Kelsee Baranowski, Peter McCloskey, Babuali Ahmed, James Legg, and David P. Hughes. Deep learning for image-based cassava disease detection. *Frontiers in Plant Science*, 8, 2017. doi: 10.3389/fpls.2017.01852. URL <https://www.frontiersin.org/journals/plant-science/articles/10.3389/fpls.2017.01852/full>.
- [10] Gnanasekaran Sambasivam and Geoffrey Duncan Opiyo. A predictive machine learning application in agriculture: Cassava disease detection and classification with imbalanced dataset using convolutional neural networks. *Egyptian Informatics Journal*, 22(1):27–34, 2021. doi: 10.1016/j.eij.2020.02.007. URL <https://www.sciencedirect.com/science/article/pii/S1110866520301110>.
- [11] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4510–4520, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html.

- [12] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 6105–6114, 2019. URL <https://proceedings.mlr.press/v97/tan19a.html>.
- [13] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. Cbam: Convolutional block attention module. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19, 2018. URL https://www.ecva.net/papers/eccv_2018/papers_ECCV/html/Sanghyun_Woo_Convolutional_Block_Attention_ECCV_2018_paper.php.
- [14] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6848–6856, 2018. doi: 10.1109/CVPR.2018.00716. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Zhang_ShuffleNet_An_Extremely_CVPR_2018_paper.html.